

Game 05 — Continuous Runner / Hurdles (Complete Plan)

Game 5 of 7 Complexity: ★★ Medium Estimate: 2–3 days Status: Not started

Complete plan (supersedes the 2026-09-09 sample). Status: **not started** — build-ready.
Slug `hurdle-runner` ; folder `apps/frontend/public/games/hurdle-runner/` .
Its engine is the foundation for Spirit Runner (07) — build to the structure in §6.

1. Overview

Endless runner: the world scrolls, hurdles approach, you jump. One input, instant restart, meters as the bragging number. Success metric: median first session ≥ 3 runs; death always reads as the player's fault.

2. Complete game spec

World & camera

- Virtual viewport 900×500 px logical, scaled to fit (letterboxed, `devicePixelRatio` aware).
- Ground line at y=420. Parallax: far hills ×0.2, near trees ×0.5, ground ×1 of scroll speed.
- Player fixed at x=225 (25 %).

Physics (fixed timestep 1/120 s, accumulator, 250 ms frame clamp)

Constant	Value	Notes
<code>SCROLL_START</code>	320 px/s	+6 px/s per second, cap 900
<code>JUMP_VY</code>	−820 px/s	on jump
<code>GRAVITY</code>	2200 px/s ²	
<code>JUMP_CUT</code>	0.55	releasing early caps upward velocity at 55 %
<code>COYOTE_MS</code>	80	grace after leaving ground
<code>BUFFER_MS</code>	100	early tap counts when landing
<code>HITBOX_INSET</code>	15 %	forgiveness on the player AABB

Obstacles (spawned ahead at x=950, culled behind x= −100)

Type	Size (w×h)	Introduced	Notes
Hurdle	24×40–60 px	0 m	low jump
Tall barrier	28×90 px	500 m	early, full-height jump

Type	Size (w×h)	Introduced	Notes
Double hurdle	two hurdles 140–180 px apart	1500 m	forces full-speed chained jumps

- **Spawner fairness:** gap between obstacles sampled 380–700 px, but never below `minGap(speed) = speed × 1.05 + 120` px (a full jump always fits). Doubles count the inner spacing toward the next gap too.
- **Collision:** AABB, player box ×0.85 inset; obstacle boxes full. Death = game over (one hit — decision master README §8 #9).
- **Score:** meters = px/10 (distance), +10 per obstacle cleared (its right edge passes player x). ❤️ pickup (P2): once per run, spawns 600–900 m; grants one extra life → on fatal hit, consume heart + 1 s invulnerability flash.

Tiers (HUD announces "Speed up!")

- Tier 1: 0–500 m (hurdles only) · Tier 2: 500–1500 m (+tall) · Tier 3: 1500 m+ (+doubles).

3. Screens & UI

State	Elements
menu	Title, best distance, control hint, ▶ Run, mute toggle
playing	Canvas world; HUD: distance, next-tier progress bar, hearts (only if ❤️ held)
paused	Overlay (auto on <code>visibilitychange</code> / ⏸)
gameover	Distance, obstacles cleared, best badge ("NEW BEST!"), [🔄 Retry] [📱 Share]

`?debug=1` query flag draws hitboxes and spawner-gap markers (dev aid, documented).

4. Controls

- **Jump:** tap anywhere / Space / ↑. `pointerdown`; hold = higher (jump cut on release).
- Coyote + buffering exactly as specced (feel-critical, in `Player.update`).
- Nothing else at MVP — one primary action. Down/fast-fall is P3.

5. Data model (localStorage)

```
game:hurdle-runner:best → { distanceM: 1204 }
game:hurdle-runner:prefs → { muted: false }
```

6. File structure & function inventory (engine is 07's base)

```
hurdle-runner/  
  index.html  
  style.css  
  
  core.js      # pure: physics step, spawner, collision, scoring – test surface  
  engine.js    # loop/camera/parallax/input (generic – 07 forks this file)  
  render.js    # draw calls: world, player, obstacles, HUD (procedural art)  
  main.js      # state machine + glue  
  config.js    # flags + strings (host-overridable); tuning constants source of truth  
  storage.js   # guarded facade: versioned best/prefs + migrations, remote adapter slot
```

`core.js` exports: `createPlayer()`, `stepPlayer(player, dt, input)` (jump/buffer/coyote/gravity), `createSpawner(rng)`, `nextSpawn(spawner, speed, lastX)` (fairness rule), `aabbHit(playerBox, obsBox, inset)`, `meters(px)`, `tierFor(m)`. `engine.js`: `createLoop` (from `shared/loop.js`), camera/parallax scroll, `visibilitychange` wiring. `render.js`: pure draw functions taking state (no logic). `main.js`: ``menu/playing/paused/gameover`` transitions.

7. Edge cases

1. Backgrounded mid-air → pause freezes physics; resume continues the same arc exactly.
2. 60 Hz vs 120 Hz displays → identical trajectories (fixed step; render reads sim state).
3. Frame spike > 250 ms → clamped (no tunneling through obstacles).
4. Jump pressed during gameover/menu → ignored (input only in `playing`).
5. ❤️ held + fatal hit → heart consumed, invulnerability ignores hits for exactly 1 s (no double-consume), HUD heart count updates.
6. Storage disabled → no best score, playable.
7. Tab-crash recovery: no mid-run persistence (stated; run is lost — acceptable).

8. Testing plan (core.js, node:test / test.html)

- Jump arc: from ground, max jump clears a 90 px tall barrier at min speed 320 px/s.
- Jump-cut: release at 50 % rise → apex lower than full hold (assert heights).
- Spawner sweep: for speed 320→900 step 20, 500 spawns each — no gap < `minGap(speed)`; doubles' inner spacing within 140–180 px.
- Collision: inset math (corner cases: 1 px overlap inside inset = no hit; at box edge = hit).
- Scoring: meters conversion; +10 awarded exactly once per obstacle.

9. Task breakdown

P0 — playable core

- ☐ `core.js` physics + spawner + collision + scoring (+ tests §8)
- ☐ `engine.js` loop, ground scroll, camera; player jump with coyote/buffer/cut
- ☐ Hurdles, speed ramp, tiers, game over; menu/gameover states; best distance

P1 — full rules & feel

- ☐ Parallax layers; procedural player run/jump animation (shapes); landing dust
- ☐ Tall barrier + double hurdles; tier announcements; WebAudio jump/land/crash + mute
- ☐ Pause overlay + `visibilitychange` (shared loop handles)

P2 — persistence/share/QA

- ☐ ❤️ pickup; share (`Ran {m} m - beat that!`); `?debug=1` hitboxes
- ☐ QA gate (master README §5): 60 fps phone check, offline, isolation greps

P3 — polish

- ☐ Day/night palette shift per 500 m; slide/duck obstacle; fast-fall; gamepad

10. Acceptance criteria

- §8 tests green; no impossible spawns (automated sweep proof).
- Identical jump arcs at 60/120 Hz (sim log comparison).
- Death never ambiguous: `?debug=1` boxes match shipped hitbox math exactly.
- Restart ≤ 1 tap, < 300 ms to first obstacle; 60 fps on a mid-range phone with particles on.

11. Deferred coupling (intentionally NOT built)

Footer/nav links, analytics events, achievements, leaderboards — per master README §7.

12. Rev 2 architecture upgrade (2026-09-11) — reference: `03-sliding-puzzle.md`

Owner-approved architecture reference for all games (Sliding Puzzle Rev 2). This game is not started, so Rev 2 is **folded directly into the build spec above** (§6 already includes `config.js` + `storage.js`) instead of being retrofitted later. Everything in §1–§11 stands; this section adds the per-game multiplayer analysis and two checklist items.

Multiplayer (per this game's nature)

- **Hot-seat:** alternate runs on one device, compare distance — trivial (a two-slot score line), no backend.

- **Async seeded track — the natural fit:** `createSpawner(rng)` already accepts a seed, so a shared seed gives both players the **identical obstacle sequence**; compare distance. Ship as `?seed=` + a share template carrying the seed (P2).
- **Live racing: gated** — real-time positions need a backend, and competitive timing requires server-side validation (client clocks can't be trusted); needs a \$7 reversal. Not planned.

Checklist additions

- ☒ P0 addition (2026-09-11, Rev 2): `config.js` (flags + per-locale strings, host-overridable) + `storage.js` facade (versioned best/prefs with legacy-key migration, remote adapter slot) — `main.js` rewired onto the facade. (2026-09-12: `config.js` wired into `main.js` — best/no-best/share/toast strings live; legacy-prefs guard fixed; unused `parallax` helper removed.)
- ☒ P2 addition (2026-09-11): Rev 2 storage landed with the facade. `?seed=` support + share template carrying the seed (async challenge) remains queued.
- ☐ Note for 07: Spirit Runner forks `engine.js` — keep the fork contract in sync when `config.js` / `storage.js` land here. (07 now carries its own `config.js` pattern.)